# A Software Engineering Framework for Reproducible Quantum Algorithm Development: Maturity Gates, Automated Validation, and Honest Limitations Across 20 Problem Domains

*This work is licensed under CC BY-NC-SA 4.0. You are free to share and adapt for non-commercial purposes with attribution.*

> ***Erratum, 2026-08-24, revised 2026-08-27 and 2026-09-25.*** *Section 4.1 of this repository copy reports the current toolchain pin, qsharp 1.31.0. The archived v2.0.0 record (DOI 10.5281/zenodo.19660251, released 2026-04-20) reports 1.27, which was accurate at that date; the repository pin moved to 1.31.0 on 2026-08-03 (#151) and the paper text was not updated at the time. Archives from v3.1.0 (DOI 10.5281/zenodo.22126595) onward report 1.31.0. The Azure Quantum runs in Section 6 were made in April 2026 and therefore used qsharp 1.27, as Section 6.1 now states.*
>
> ***Erratum, 2026-08-28.*** *Sections 7.1 and 9 and the conclusion of the archived v1.0.1 and v2.0.0 records state that all 20 problems had reached Stage C and that four had reached Stage D. That was not accurate at either date: when v2.0.0 was archived on 2026-04-20,* `docs/objective-kpis.json` *recorded 3 problems at Stage C, 17 at Stage B and none at Stage D. The first Stage D promotion landed on 2026-05-06 (#89), after both archives. This repository copy now states the figures that file records - 9 at Stage C, 8 at Stage B, 3 at Stage D - and* `tooling/test_doc_claims.py` *fails the build when the two disagree. A paper arguing that maturity gates prevent premature claims should not have overstated its own.*
>
> ***Revision, 2026-09-25, archived as v3.2.0 on 2026-09-26.*** *Every number, citation and description in this copy was checked against the repository's committed data and code. The following did not hold and are corrected in the text: the cross-platform agreement in Section 7.2 (the committed histograms give the same unique most frequent outcome for 14 of 19 problems, not 17); the reading of the noise table in Section 7.3.1 (its columns are single-qubit error rates, with two-qubit rates ten times higher, and its first noisy column now*

*matches the committed model output); the AES-128 Grover cost in Section 7.3 (about 10^19 is the iteration count, not the gate count); Section 7.4, which now states that the Goemans-Williamson column is a computed bound and that GW was not run; the utility-scale classification in Section 8, which contradicted both the framework it cites and the project's own screening; the description of CI enforcement in Sections 1.1 and 5 (the checks fail the build but do not block merges, and verify a declared stage and two README sections, not the evidence each stage requires; the documentation-claim guards were not run by CI at all until a step was added in this revision); the toolchain, kernels and four qubit counts behind the Section 6.1 syntax checks; the kernel behind the largest resource estimate and the stale range in Lesson 3 of Section 7.5; the stages in Appendix A; and several related-work descriptions and citations. Section 8 adds a limitation found during this check, that stage labels rested on evidence nothing verified, and following it up found defects in the kernels themselves. The five QPE kernels applied only diagonal rotations, so they returned the diagonal energy of their input basis state rather than a ground-state energy; the quantum-walk kernel's position never depended on its coin; and Shor's order finding did not find the period, because its modular multiplier was right for two of its bases and its inverse QFT was wrong, so the demonstration Section 7.2 said factored 15 usually did not, and its resource estimate used a base that shares a factor with 15 and so contained no modular arithmetic. Those kernels were rewritten and tested against exact simulation (Section 4.2), the resource estimates now target the rewritten routines (Section 4.4), every calibration ensemble was regenerated, and CI now fails when the ensemble behind a Stage C or D label does not match the sources it claims to describe (Section 5.2). Section 9 also records that Part 6 of the lecture series [15], on cost, has been published; it had been listed as forthcoming.*

***Revision, 2026-09-26, after the v3.2.0 archive.*** *A second audit compared the remaining kernels with exact simulation and their documentation with their code. Canonical amplitude estimation in the risk-analysis problem reflected about $A^\dagger|0\rangle$ instead of $A|0\rangle$ and omitted the $-1$ that gives the controlled Grover iterate its eigenphases $\pm 2\theta$, so its phase register peaked at the wrong outcomes; its one sanity test used a self-inverse state preparation at $a = 1/2$, where neither fault shows. Its published tail probability matched no computation of the model (the circuit encodes 16.14%), its ensembles came from the faulty kernel, its calibration read the phase register in the wrong bit order, and its IQAE driver was not the algorithm of Grinko et al. [22] that it cited. The kernel now follows Theorem 11 of Brassard et al. [21] exactly, the driver implements Grinko et al.'s Algorithm 1 and its intervals hold their stated confidence over repeated runs, and IQAE is compared with Monte Carlo on the same distribution at equal precision (Section 7.2). The high-frequency-trading problem, listed as amplitude estimation for Value at Risk, never implemented amplitude estimation: it samples a marked-*

> *state probability directly. The two HHL kernels were not HHL and have been rebuilt as the textbook circuit [23], which reproduces $A^{-1}b$ exactly for the 2×2 diffusion matrix and within its clock resolution for the other system. The QAOA calibrations and estimates ran fixed angles (an expected cut of 0.13 on a triangle, worse than a random cut) and now use grid-searched angles on the models the demonstrations build; the protein-folding kernel is a four-variable toy QUBO and is now described as one, and three QAOA hardware kernels (08, 12, 20) implemented other models than the programs they stood for. The swap-test encoding dropped signs and its hardware kernel compared unrelated states. Section 4.3's statement that every classical baseline is the standard approach for its domain did not hold for eight problems. Resource estimates now count Toffoli gates, which were previously dropped (10,687 in the risk-analysis kernel), and Lesson 3 is recomputed; the estimator summaries of the archived problems had printed a mock constant as their estimate. Under the pinned QDK one hardware kernel no longer compiled to QIR, and CI now compiles all twenty. The noise study in Section 7.2 was regenerated for the current kernels; the emulator histograms still describe the kernels of April 2026. Counting both revisions, 16 of the 20 kernels (01, 02, 03, 04, 06, 07, 08, 09, 11, 12, 13, 14, 17, 18, 19, 20) either did not compute what they were described as computing or were mislabelled, although every one of them compiled and passed CI.*
>
> ***Revision, 2026-10-18.*** *Lesson 3 of Section 7.5 gave the share of physical qubits held by T factories without naming the factory model behind it. It now says the shares come from QRE v3's round-based distillation factory, the only one these estimates use, and that magic state cultivation [25, 26] is not modelled; a cultivation scenario is tracked in #296. References [25] to [27] are new. No estimate changed.*

## Authors

Werner Rall

## Abstract

We present a software engineering framework for organizing quantum algorithm development across multiple problem domains. The framework provides standardized project structure, automated CI/CD validation, and a four-stage maturity gate model designed to prevent premature quantum advantage claims. We apply the framework to 20 problem domains using Microsoft Q#, implementing ten algorithm families at toy scale (2-14 qubits) where classical simulation is trivial. **We do not claim quantum**

**advantage for any problem.** All implementations use small, highly structured instances specifically chosen for correctness validation, not for demonstrating quantum utility. Tests against exact simulation confirm that each kernel implements its stated algorithm at small scale; writing them showed that 16 of the 20 kernels, all of which compiled and passed CI, either did not compute what they were described as computing or were mislabelled. None of this reveals anything about practical scalability. The primary contribution is the methodology itself particularly the maturity gate model that forces honest assessment of what has and has not been demonstrated not the individual quantum implementations. Screening the same 20 problems against utility-scale filters archived eleven of them, including all three that had completed the Stage D evidence package, which shows that documenting a claim fully and the claim being favorable are independent. We include a scaling analysis for Grover search showing the expected $O(\sqrt{N})$ query complexity alongside the corresponding growth in gate count and the effect of gate noise under an explicit analytic model. The framework and all implementations are open-source (DOI: 10.5281/zenodo.19222020).

**Keywords:** quantum software engineering, development methodology, maturity gates, reproducibility, Q#, CI/CD

## 1. Introduction

The quantum computing community faces a credibility challenge. Claims of quantum advantage have repeatedly been weakened or overturned: sampling and utility-scale experiments have been reproduced by improved classical simulation [8, 9], a claimed exponential speedup for recommendation systems was matched by a quantum-inspired classical algorithm [10], and accounting for the constant-factor cost of quantum error correction erodes quadratic speedups [6]. A contributing factor is the absence of standardized development practices that enforce honest assessment at each stage of quantum algorithm development.

This paper addresses not the algorithms themselves which are well-established in the literature but the **software engineering process** of developing, validating, and honestly assessing quantum implementations. We propose a framework that applies the same rigor to quantum development that software engineering applies to classical systems: standardized structure, automated testing, evidence-based quality gates, and reproducible benchmarks.

**Scope and limitations.** Our 20 implementations are toy-scale (2-14 qubits) where classical simulation is trivial. We use standard textbook algorithms applied to small structured instances. We do not advance quantum algorithm theory, discover new speedups, or demonstrate quantum utility. The contribution is purely methodological: a

framework designed to prevent the common failure mode of claiming more than the evidence supports.

### 1.1 Contributions

1. **Maturity Gate Model**: A four-stage procedural framework (A→D) with explicit evidence requirements at each stage, designed to prevent premature quantum advantage claims. This is a process contribution, not a complexity-theoretic one.
2. **Standardized Problem Structure**: A reproducible template ensuring every problem includes classical baselines, parameterized instances, quantum implementations, and honest limitations documentation.
3. **Automated Policy Checks**: CI jobs that fail when a problem's declared maturity stage falls below its policy minimum or required README sections are missing. Section 5 states what these checks do and do not verify.
4. **Scaling Analysis**: For one representative problem (Grover search), we show how gate count and noise sensitivity grow with problem size under an explicit analytic model, illustrating the gap between toy validation instances and practically relevant scales.
5. **Lessons Learned**: Practical insights from building 20 quantum implementations, including how compiled-but-incorrect code, encoding bugs, mock estimates and presence-only checks can create false confidence.

## 2. Related Work

Prior benchmarking efforts include the QED-C application-oriented benchmark suite [1], which measures how faithfully hardware executes many well-known algorithms as circuit width and depth grow, and the QUARK framework [2], which defines application-level benchmarks, initially for industrial optimization problems. Software development kits such as Qiskit [3] provide the circuit construction and compilation layer that such benchmarks exercise. Our work differs in emphasis rather than breadth: instead of measuring hardware or software performance, it defines process gates that govern what may be claimed about an implementation, and it emphasizes reproducibility infrastructure. The utility-scale screening in Section 8 operationalizes the argument of Hoefler, Häner and Troyer [11] that quadratic speedups and data-intensive problems are unlikely to yield practical advantage.

## 3. Framework Design

### 3.1 Problem Structure

Each of the 20 problems follows an identical directory layout; the eleven archived problems (Section 8) keep the same layout under `problems/archived/`:

```
problems/XX_name/
├── qsharp/          # Q# quantum implementation (src/Main.qs, Hardwa
├── circuits/        # Resource estimate (estimate.json) and circuit
├── python/          # Classical baseline and analysis
├── instances/       # Parameterized YAML configurations (small/mediu
├── estimates/       # Resource estimation artifacts
├── plots/           # Generated visualizations
├── Makefile         # Reproducible build commands
└── README.md        # Documentation with advantage claim contract
```

This uniformity enables automated tooling to operate across all 20 problems without problem-specific logic.

## 3.2 Maturity Gate Model

We define four progressive stages, each with explicit evidence requirements:

**Stage A (Classical Baseline Validated):** - Deterministic classical baseline implementation - Instance coverage (small, medium at minimum) - Reproducible command path documented - Metrics persisted in standardized JSON schema

**Stage B (Quantum Implementation Ready):** - Q# quantum algorithm implemented with real gate operations - Build and run validation passing in CI - Resource estimation path documented - Algorithm assumptions and known limits stated

**Stage C (Hardware-Aware Validation):** - Multi-run calibration with uncertainty bounds - Quantum/classical comparison with confidence intervals - Hardware mapping and transpilation assumptions documented - DiVincenzo readiness criteria assessed

**Stage D (Advantage Evidence Package):** - Advantage claim template completed with explicit category (theoretical/projected/demonstrated) - Fair classical comparator identified and justified - Residual risks and assumption sensitivities documented - Reproducible end-to-end evidence chain

Stage D certifies that a claim is completely documented, whatever its category; a Stage D problem may, and in this project does, carry a claim that no practical advantage has been shown.

**How stages are recorded.** Each problem's README declares its current stage on a "Current gate" line. The policy check (Section 5.2) verifies that the declared stage meets a per-problem minimum and that every README contains the advantage-contract and

DiVincenzo-readiness sections; for one problem (05) it also requires the estimator-summary and backend-assumption files. It does not inspect the calibration ensembles or evidence packages that Stages C and D require, and it does not re-derive the stage from the evidence, so a declaration is accepted whether or not that evidence exists or describes the current kernel. Section 8 (limitation 8) describes where this mattered.

### 3.3 Advantage Claim Contract

Each problem includes a structured advantage claim with fields for: claim category, problem class and regime, baseline algorithm and fairness rationale, quantum resource scaling claim, data-loading and I/O assumptions, noise/error model assumptions, confidence/uncertainty method, and residual risks. This prevents implicit or unsupported advantage claims.

### 3.4 DiVincenzo Readiness Overlay

For Stage C/D problems, we assess the five DiVincenzo criteria for quantum computation [12]: scalable qubit system, initialization capability, coherence relative to gate times, universal gate set, and qubit-specific measurement. The criteria describe a physical platform rather than an algorithm, so the assessment records how well the hardware assumed by each problem's resource estimate meets them. Each criterion is tagged as met, partial, not-yet, or not-applicable with evidence notes.

## 4. Implementation

### 4.1 Toolchain

- **Quantum:** Microsoft Q# with modern QDK (currently pinned to qsharp 1.31.0, Python-hosted, no .NET dependency; the April 2026 Azure runs in Section 6 used qsharp 1.27)
- **Classical:** Python 3.11 with NumPy, SciPy, Matplotlib
- **Cloud:** Azure Quantum with Quantinuum and Rigetti providers
- **CI/CD:** Eight GitHub Actions workflows, three of which run on pull requests (Section 5.1)
- **Website:** Statically generated Next.js dashboard

### 4.2 Algorithm Families

Table 1 summarizes the ten algorithm families implemented across the 20 problems. Qubit counts cover the hardware kernels, the demonstration instances in each `Main.qs`, and the instances passed to the resource estimator.

| Algorithm Family | Problems | Qubits | Key Operations |
|---|---|---|---|
| VQE, then QPE* | 01, 02, 07, 14, 17 | 2-14 | Ry, CNOT, Rz, Pauli measurement (VQE); Trotterized Pauli exponentials, phase estimation (QPE) |
| QAOA | 05, 08, 12, 20 | 3-4 | H, ZZ cost (CNOT+Rz), Rx mixer |
| Grover | 10, 15 | 3-12 | Oracle marking, diffusion operator |
| HHL | 04, 13 | 5-7 | Controlled exp(iAt), QPE, controlled Ry(2 arcsin(C/λ)), inverse QPE, post-selection |
| Amplitude estimation† | 03 | 5-14 | State prep, oracle, Grover^k; canonical QAE adds a QPE precision register, IQAE does not |
| Shor | 09 | 8 | QPE, controlled modular multiply |
| Swap Test | 11 | 5 | Controlled-SWAP, Hadamard test |
| QEC | 16 | 5 | Syndrome extraction, correction |
| Quantum Walk | 18 | 3 | Coin rotation, conditional shift |
| Trotter | 19 | 2-4 | ZZ couplings, transverse field (an Ising chain) |

* The VQE routines (2-3 qubits) remain in each `Main.qs`. On 2026-04-18 (#66) the hardware kernels of these five problems were replaced by quantum phase estimation (QPE) kernels, but every controlled evolution in them was built from Rz and CNOT-conjugated Rz gates, a diagonal unitary: from a computational basis state the phase register returned that state's diagonal energy, never the ground-state energy. On 2026-09-25 they were rewritten as Trotterized QPE on a 4-qubit Jordan-Wigner two-site Hubbard model (01), the parity-reduced 2-qubit STO-3G $H_2$ Hamiltonian (02), the 2-qubit deuteron model of Dumitrescu et al. [20] (17), and 2-qubit models labelled as toys (07, 14), and `tooling/test_qpe_kernels.py` now compares each against exact diagonalization of the same Hamiltonian. The hardware kernels use 2 phase bits (4-6 qubits) and the resource estimates 10 (12-14 qubits). The emulator histograms and the noise study in Section 7.2 were produced by the VQE kernels and predate both changes; the calibration ensembles and resource estimates were regenerated from the rewritten QPE routines. The same revision found that the quantum-walk kernel (18) swapped its position qubits whichever way the coin pointed, and rewrote its shift as a coin-controlled step ( `tooling/test_exciton_walk.py` ). It also found that Shor's order finding (09) did not find the period. Its mod-15 multiplier was right for a = 2 and 4 only, its inverse

QFT processed the qubits in the wrong order with angles twice too large, and its hardware kernel applied one multiplication controlled by the wrong qubit. Its resource estimate ran base 3, which shares a factor with 15, so the multiplier applied nothing and the estimate contained no modular arithmetic. All four were corrected and are tested against the exact period of every base coprime to 15 (`tooling/test_shor_kernel.py`); the estimate now runs base 7. The Trotter kernel (19) was found to compute what its code says but not what it was called: it evolves a transverse-field Ising chain, which has no gauge fields and maps to free fermions, and the quantity described as a Wilson loop is the product of Z over all sites. It is now described as a spin-chain stand-in and checked against exact evolution (`tooling/test_ising_chain_kernel.py`).

On 2026-09-26 the remaining kernels were compared with exact simulation. Canonical QAE (03) built its reflection as $A^\dagger S_0 A$ instead of $AS_0A^\dagger$ and omitted the $-1$ in $Q = -AS_0A^\dagger S_\chi$, which becomes a relative phase once Q is controlled, so its phase register peaked at 0 and 32 of 64 where Theorem 11 of Brassard et al. [21] puts the peaks at 8 and 56; `tooling/test_qae_kernel.py` now compares the register with that theorem from the state vector, and the IQAE driver implements Algorithm 1 of Grinko et al. [22]. The HHL kernels (04, 13) used ad hoc rotations and a per-bit inversion; they are now the textbook circuit [23], with exact controlled evolution and a $C/\lambda$ inversion, and `tooling/test_hhl_kernel.py` ties their outputs to a direct solution of $Ax = b$. The swap-test encoding (11) dropped the signs of its amplitudes, and three QAOA hardware kernels implemented other models than the programs they stood for: 08 and 12 ran chains with equal couplings instead of their weighted QUBOs, and 20 used different biases and two of its three couplings (`tooling/test_swap_test_kernel.py`, `tooling/test_qaoa_kernels.py`).

† Problem 06 was listed in this row until 2026-09-26. Its kernel prepares a 2-qubit state and measures a marker once per shot, sampling a marked-state probability directly; it never implemented amplitude estimation, so it is not counted as a family.

### 4.3 Classical Baselines

Every problem includes a deterministic classical baseline, and all baselines produce JSON output conforming to a shared schema. Each was meant to be both the standard classical approach for its domain and the comparison target for its quantum kernel, as exact diagonalization is for the QPE models and brute-force enumeration is for MaxCut at small scale. For eight problems it is not both. The trading baseline (06) is a moving-average strategy unrelated to the kernel's marked-state probability; the protein-folding (08) and mission-planning (20) baselines are heuristic scoring scripts and the scheduling baseline (12) a greedy tardiness heuristic, none of which evaluates its kernel's QUBO; the climate baseline (13) is an energy-balance model of a different system; the machine-learning baseline (11) is kernel ridge classification, a different task from the swap test's

overlap estimate; the cryptography baseline (10) is a toy lattice-attack cost formula, while the key-search kernel's comparator is exhaustive search; and the Trotter baseline (19) is a heuristic formula rather than the exact solution of the Ising chain, which is classically easy. The risk-analysis Monte Carlo sampled the continuous model rather than the circuit's 16-level discretization until a comparison on the same distribution was added in September 2026. Until then this section described every baseline as the standard approach for its domain; each README now says what its baseline computes.

### 4.4 Resource Estimation Pipeline

We employ a centralized estimation manager ( `tooling/generate_estimates.py` ) that runs the Quantum Resource Estimator v3 ( `qdk.qre` ) [19] on each problem's estimation entry point. All 20 problems have real profiles targeting a gate-based superconducting architecture (50 ns gates, $10^{-3}$ error rate) with a surface code, round-based magic-state factories and a total error budget of $10^{-3}$. QRE v3 returns a Pareto frontier trading physical qubits against runtime; we report the point with the fewest physical qubits, which is also the slowest. The estimated programs are the toy instances themselves, compiled for fault-tolerant execution, not utility-scale versions of the problems; for the five QPE problems of Table 1 they are the QPE routines with a 10-bit phase register. Physical qubit counts range from 1,746 (3-qubit QEC repetition code) to 754,715 (QPE on the materials-discovery model). The pipeline also supports batch calibration via `tooling/generate_calibration_ensemble.py`, which runs each kernel 20 times to produce ensemble statistics with 95% confidence intervals (for the QPE problems each run is the most frequent of 8 estimates at 8 phase bits). The intervals are normal approximations: for four problems (02, 05, 16 and 17) every run returned the same value, the interval has zero width and does not bound the mean, and the site reports it as undefined. All 20 committed ensembles now carry valid statistics and record hashes of the Q# sources they ran; before the 2026-09-25 revision 10 of the 20 did (Section 8, limitation 8). Until 2026-09-26 the estimates reported no Toffoli gates at all: the trace reports them as CCX, which the summary did not map, so the risk-analysis kernel's 10,687 Toffolis went uncounted.

The QRE v3 migration superseded an earlier range of 1,764 to 401,400 produced by the now-deprecated `qsharp.estimate` API. For the programs estimated at the time, under matched assumptions, QRE v3 reported 1.0× to 3.9× fewer physical qubits at 1.5× to 2× the runtime, consistent with selecting the low-qubit corner of the frontier rather than any change in the underlying circuits.

Current target architectures: - `qubit_gate_ns_e3` : Gate-based model, 50 ns gate time, $10^{-3}$ error rate - `qubit_gate_ns_e4` : Gate-based model, 50 ns gate time, $10^{-4}$ error rate - `qubit_gate_us_e3` : Gate-based model, 100 µs gate time, $10^{-3}$ error rate - `qubit_gate_us_e4` : Gate-based model, 100 µs gate time, $10^{-4}$ error rate

The ns profiles are typical of superconducting qubits and the µs profiles of trapped ions, but the four profiles differ only in gate time, measurement time and physical error rate ( `tooling/estimator_config.py` ).

Majorana profiles and the floquet code were retired at the QRE v3 migration: QRE v3 ships no floquet code, and the Majorana architecture yielded an empty Pareto frontier for every code and factory combination tested.

## 5. Automated Validation

### 5.1 CI Pipeline

Three workflows run on pull requests; the dependency-graph check runs only when code, workflow or build files change:

1. **CI/CD**: its build-and-test job validates workflow syntax, compiles and runs all 20 Q# projects, runs the classical baseline tests and the evaluator test suite, validates JSON schemas, enforces the maturity policy (Section 5.2), and runs the tests under `tooling/` and `viz/` , which include the documentation-claim, evidence-provenance and kernel-correctness checks. That last step was added on 2026-09-25; until then CI did not run those guards, so the stage-count check cited in the errata above failed only when someone ran it locally
2. **Dependency Graph Drift Check**: fails when the committed dependency and reachability map is stale
3. **Azure Secret Hygiene**: prevents accidental credential commits

Five further workflows deploy the website and the evaluator API, run a nightly resource-estimation sweep, KPI refresh and runnable-correctness audit, schedule an uptime probe of the deployed API (its cron asks for every 30 minutes; GitHub's throttling of scheduled workflows has produced roughly one run every four hours), and, since September 2026, run a weekly check that the deployed evaluator's Q# generation returns programs that compile and do quantum work. Earlier versions of this paper listed seven pull-request checks, including dedicated Stage D readiness, runnable-correctness and reporting-integrity gates; these were consolidated or removed between April and June 2026 (#67, #124), and the correctness audit now runs nightly rather than per pull request. None of the checks is configured as a required status check on the main branch, so a failing check reports a problem but does not block a merge.

### 5.2 KPI Dashboard

An automated reporting script ( `stage_kpis.py` ) scans all 20 problem directories and produces a maturity status report with four flags per problem: advantage contract

present, estimator profile summary generated, backend assumptions documented, and DiVincenzo readiness assessed. The build-and-test job fails when a declared stage is below its per-problem minimum, when a README lacks the advantage-contract or DiVincenzo-readiness section, or, for 05 only, when the estimator-summary or backend-assumption file is missing. These are presence checks (a heading in the README, or an artifact file with non-placeholder content): they establish that some evidence was filed, not that it is current, correct, or the evidence the declared stage requires. Two checks added on 2026-09-25 go further. At Stage C and above the calibration ensemble must have at least 20 runs and complete statistics, and the hashes it records of the Q# sources it ran must match the current sources, so rewriting a kernel invalidates its ensemble. At Stage D the claim category in the README must match the machine-readable contract.

Current coverage: 20/20 problems have advantage contracts, estimator summaries, backend assumptions and DiVincenzo readiness sections.

## 6. Azure Quantum Integration

### 6.1 Circuit Validation

On 12-13 April 2026, the hardware kernels of all 20 problems were compiled from Q# to QIR with the modern QDK (qsharp 1.27, the version pinned at the time) and accepted by the Quantinuum H2-1SC syntax checker via Azure Quantum. A syntax checker verifies that a program is well formed for the target; it does not simulate it. Table 2 lists the kernels as recorded then, with approximate gate counts: 20 kernels from 19 problems (two for 01). The 04_linear_solvers kernel also passed and is not listed. The VQE kernels in the table (six rows covering five problems) were replaced by QPE kernels five days later (Section 4.2); those replacements were submitted to the H2-1E emulator on 10 June 2026, but their results were never retrieved into the committed run history, and they were themselves rewritten on 2026-09-25 because their evolution was diagonal. The quantum-walk and Shor kernels listed here are also the versions before their 2026-09-25 corrections, and the QAE (03), HHL (04, 13), swap-test (11) and QAOA (05, 08, 12, 20) kernels were corrected or aligned with their estimated programs on 2026-09-26 (Section 4.2). None of the rewritten kernels has been submitted to Azure Quantum. Under the currently pinned qsharp 1.31.0 the QAE kernel no longer compiled to QIR (the compiler panicked on partially applied operations inside controlled calls); it was restructured, and CI now compiles all twenty hardware kernels for the Adaptive_RI profile ( `tooling/test_hardware_kernels_compile.py` ).

| Circuit | Algorithm | Qubits | Gates | Status |
|---|---|---|---|---|
| Hubbard VQE (ZZ basis) | VQE | 2 | ~8 | Succeeded |

| Circuit | Algorithm | Qubits | Gates | Status |
|---|---|---|---|---|
| Hubbard VQE (XX basis) | VQE | 2 | ~10 | Succeeded |
| MaxCut QAOA depth-1 | QAOA | 3 | ~12 | Succeeded |
| Database Grover 4-qubit | Grover | 4 | ~200 | Succeeded |
| Key Search Grover | Grover | 3 | ~150 | Succeeded |
| Scheduling QAOA | QAOA | 4 | ~24 | Succeeded |
| QEC Repetition Code | QEC | 5 | ~10 | Succeeded |
| Shor N=15 (a=7) | Shor | 8 | ~50 | Succeeded |
| HHL Diffusion PDE | HHL | 5 | ~30 | Succeeded |
| QAE Simplified | QAE | 5 | ~20 | Succeeded |
| Quantum VaR (direct sampling; not QAE) | Sampling | 3 | ~10 | Succeeded |
| QAOA Protein Folding | QAOA | 3 | ~30 | Succeeded |
| VQE Catalysis | VQE | 2 | ~6 | Succeeded |
| VQE Drug Binding | VQE | 2 | ~8 | Succeeded |
| Swap Test Kernel | Swap | 5 | ~10 | Succeeded |
| VQE Band Gap | VQE | 2 | ~6 | Succeeded |
| VQE Deuteron | VQE | 2 | ~8 | Succeeded |
| Quantum Walk Exciton | Walk | 3 | ~8 | Succeeded |
| Trotter Gauge | Trotter | 4 | ~40 | Succeeded |
| QAOA Trajectory | QAOA | 3 | ~30 | Succeeded |

### 6.2 Shared Azure Workflow

A problem-agnostic Azure submission pipeline ( `tooling/azure/smoke_problem.py` ) enables any problem to submit circuits to Azure Quantum through a single interface, with env-gated authentication, manifest tracking, and audit reporting.

## 7. Results and Discussion

### 7.1 What the Framework Demonstrates (and What It Does Not)

Of the 20 problems, 9 have reached Stage C (declared on the basis of calibration ensembles and real Azure Quantum Resource Estimator profiles; limitation 8 in Section 8 records what that evidence does and does not establish), 8 remain at Stage B, and **3 have reached Stage D** (advantage evidence packages with explicit claim contracts). The 3 Stage D problems (QAE risk analysis, QAOA MaxCut, Grover database search) filed claim contracts in the categories "theoretical" (QAOA MaxCut and QAE risk analysis) and "projected" (Grover database search); until this revision the QAE README gave "projected", and CI now fails when a README and its contract disagree. None claims a demonstrated advantage. The measured ensembles in the QAE package came from the canonical kernel before its correction on 2026-09-26 and are superseded by exact tests of the kernel, by IQAE runs compared with Monte Carlo on the same distribution, and by a regenerated calibration ensemble. The QAOA MaxCut contract states that there is "no proven quantum advantage for MaxCut QAOA at any constant depth" and identifies the Goemans-Williamson 0.878-approximation [7] as the polynomial-time classical competitor. The Grover contract rests on the provably optimal O(√N) query speedup and lists oracle synthesis overhead as its main residual risk. It does not establish a problem size at which the speedup becomes practical, and for unstructured data the cost of loading the database into an oracle or QRAM is itself O(N).

Maturity and merit are independent. When the utility-scale filters of Section 8 were applied to all 20 problems in April 2026 (five filters then; the sixth, F6, was added in August 2026), eleven were archived. They include all three Stage D problems: QAE risk analysis for a quadratic speedup offset by data-loading cost, QAOA MaxCut for having no proven speedup, and Grover database search for a quadratic speedup offset by QRAM cost. The nine active problems are all at Stage C. A complete evidence package documents a claim; it does not make the claim favorable.

**What our results demonstrate:** - Algorithmic correctness at small scale (Grover finds marked items, order finding returns the period of every base coprime to 15, QEC corrects single bit-flips, canonical QAE's phase register follows Theorem 11 of Brassard et al. [21], HHL returns $A^{-1}b$ to within its clock resolution, which each added clock bit halves (exactly for the 2×2 diffusion matrix), and the QPE, quantum-walk, QAOA and swap-test kernels sample the distributions that exact simulation of the same circuits predicts) - The framework's ability to organize and validate quantum development across multiple problem domains - Practical lessons about failure modes in quantum software development

**What our results do NOT demonstrate:** - Quantum advantage over classical methods for any problem - Noise resilience or hardware viability - Scalability to practically relevant problem sizes - Superiority over state-of-the-art classical algorithms

## 7.2 Simulator and Emulator Results Are Sanity Checks, Not Experiments

No problem was run on quantum hardware. Validation used noiseless local simulation, the Quantinuum H2-1SC syntax checker, the Quantinuum H2-1E emulator (which applies a noise model of the H2 hardware) and the Rigetti QVM simulator. Zero-variance results (e.g., 100% QEC correction rate, exact QAOA optimality) confirm correct circuit construction but reveal nothing about performance under realistic noise, which limits the size of circuit that near-term devices can execute reliably [5]. A circuit that achieves perfect results on a simulator may fail completely on hardware with $10^{-3}$ two-qubit gate error rates once it is deep enough.

The H2-1SC syntax checker validates that circuits are well-formed QIR but does not simulate quantum behavior. In April 2026, H2-1E emulator results were collected for all 20 problems (100 shots each), and 19 were also run on the Rigetti QVM. For 14 of those 19, the most frequent outcome is the same unique outcome on both platforms; for two more (05, 08) the QVM histogram has a tie that includes the H2-1E mode; for three (03, 12, 20) the modes differ. The total variation distance between the paired 100-shot histograms ranges from 0.00 to 0.44 (median 0.06). With 100 shots, sampling error alone is up to ±0.05 in each outcome probability (one standard deviation), so these comparisons show that both platforms compiled and executed the same programs; they are not evidence about hardware behavior. They ran the kernels of April 2026: for 01, 02, 07, 14 and 17 the VQE kernels, for 18 a walk whose position never moved, for 09 an order-finding kernel that did not find the period, for 03 a QAE kernel with the wrong Grover iterate, for 04 and 13 kernels that were not HHL, for 11 a swap test between unrelated states, and for 05, 08, 12 and 20 QAOA kernels that differed from the programs now estimated (Section 4.2).

A separate local simulation with the QDK's depolarizing noise model at strengths p = 0.001, 0.01 and 0.05 (100 shots each) measures the classical fidelity between each kernel's noisy and ideal output histograms. It was regenerated for the current kernels on 2026-09-26; the April 2026 run had used the kernels described above. At p = 0.05 the fidelities range from 0.12 (Grover database search with three iterations) to 1.00 (the QEC kernel, whose output is its corrected bit), with the five QPE kernels at 0.39-0.84, Shor at 0.53 and the quantum walk at 0.49. These estimates are shot-noise limited: several rise between p = 0.001 and p = 0.01, which added noise cannot cause. A September 2026 run of the Grover database-search kernel (four qubits, three iterations) on H2-1E returned the marked item in 80 of 100 shots, against an ideal 96.1% and about 6 in 100 for random guessing ( `problems/archived/15_database_search/azure_runs/` ).

Selected correctness validation results: - **Grover (10_pqc)**: 80-94% success in noiseless simulation of the 3-, 4- and 5-qubit instances, consistent with the ideal 78%, 91% and 90% for the iteration counts the implementation uses (1, 2 and 3, one fewer than the optimal counts used in Section 7.3) - **Shor (09_factorization)**: Order finding for N = 15 returns s × 16 / r for every base coprime to 15, and the classical step recovers 15 = 3 × 5. Earlier versions of this paper said the implementation correctly factored 15; it did not.

The counting register was nearly uniform, and in three runs of the demonstration the most frequent candidate period was 8 and no factor was found in 8 of 9 attempts (Section 4.2). The controlled modular multipliers are fixed permutations compiled for N = 15, so the circuit demonstrates the structure of period finding, not scalable modular arithmetic; compiled demonstrations of this kind say little about factoring in general [16]. It is a textbook demonstration, not a research contribution - **QAE (03_qae_risk)**: the canonical kernel's phase register matches Theorem 11 of Brassard et al. [21] to $10^{-9}$. IQAE on the kernel returned [0.152, 0.191] around the encoded a = 0.1614 from 600 applications of the state preparation, where Monte Carlo on the same distribution needs 1,373 samples for the same half-width; at half-width 0.0004 the counts are 42,715 and 3.2 million. These are noiseless query counts, not run times - **HHL (04_linear_solvers, 13_climate_modeling)**: on the 2×2 diffusion matrix the post-selected output equals $|A^{-1}b|^2$ = [0.8, 0.2] with success probability 5/9, the textbook values for C = 1; for the other system, whose eigenvalues fall between clock values, the fidelity with $A^{-1}b$ rises from 0.99957 with 3 clock bits to 0.99998 with 4 and 0.99999 with 5 - **QEC (16_error_correction)**: 512/512 = 100% correction on simulator (meaningless without noise the entire point of QEC is noise resilience)

## 7.3 Scaling Analysis: Grover Search (Computed)

To illustrate the gap between toy validation and practical utility, we computed Grover search resource requirements at increasing qubit counts with an analytic gate-count model ( `tooling/run_paper_analyses.py` ). Each iteration applies two multi-controlled Z gates (oracle and diffusion), each counted as n-2 Toffoli gates, and each Toffoli is decomposed into 6 CX and 7 T gates [4]; n is the search-register size and the iteration count is the standard $\lfloor(\pi/4)\sqrt{N}\rfloor$. The model omits ancilla qubits and their uncomputation (a textbook V-chain construction of an n-qubit multi-controlled Z uses 2n-5 Toffolis and n-3 ancillas), so its counts are order-of-magnitude illustrations, not compiled circuits.

| Qubits (n) | Keyspace (N) | Iterations | CX Gates | T Gates | Total Gates | Success (noiseless) |
|---|---|---|---|---|---|---|
| 3 | 8 | 2 | 24 | 28 | 84 | 94.5% |
| 4 | 16 | 3 | 72 | 84 | 222 | 96.1% |
| 5 | 32 | 4 | 144 | 168 | 424 | 99.9% |
| 6 | 64 | 6 | 288 | 336 | 828 | 99.7% |
| 8 | 256 | 12 | 864 | 1,008 | 2,424 | 100% |
| 10 | 1,024 | 25 | 2,400 | 2,800 | 6,650 | 100% |

| Qubits (n) | Keyspace (N) | Iterations | CX Gates | T Gates | Total Gates | Success (noiseless) |
|---|---|---|---|---|---|---|
| 12 | 4,096 | 50 | 6,000 | 7,000 | 16,500 | 100% |
| 16 | 65,536 | 201 | 33,768 | 39,396 | 92,058 | 100% |
| 20 | 1,048,576 | 804 | 173,664 | 202,608 | 471,144 | 100% |

The quadratic speedup ($O(\sqrt{N})$ queries) is real in query complexity, but total gate count grows as $O(\sqrt{N} \times n)$ where n is the number of qubits. For n=20 ($N \approx 10^6$), the model gives ~471K gates. For a 128-bit keyspace it gives about $1.4 \times 10^{19}$ Grover iterations and $6 \times 10^{22}$ gates with a trivial marking oracle; with a reversible AES-128 circuit as the oracle, published estimates reach about $2^{86} \approx 8 \times 10^{25}$ T gates on 2,953 logical qubits [13], far beyond any foreseeable hardware. **No amount of CI/CD rigor changes these scaling realities.**

### 7.3.1 Noise Impact on Grover Scaling

Using the same gate counts and a simple survival model, in which the circuit returns the noiseless output distribution if no counted gate fails and a uniformly random output otherwise, we computed the impact on Grover success probability. Each of the 6n-2 H and X gates per iteration fails with probability $p_1$ and each CX with $p_2 = 10p_1$; the T gates inside the Toffolis are not counted as failure points, which makes the model optimistic:

| Qubits | 2Q Gates | Noiseless | $p_1$=0.1%, $p_2$=1% | $p_1$=1%, $p_2$=10% | $p_1$=5%, $p_2$=50% |
|---|---|---|---|---|---|
| 3 | 24 | 94.5% | 74.9% | 17.2% | 12.5% |
| 4 | 72 | 96.1% | 47.1% | 6.3% | 6.2% |
| 5 | 144 | 99.9% | 23.5% | 3.1% | 3.1% |
| 6 | 288 | 99.7% | 6.0% | 1.6% | 1.6% |
| 8 | 864 | 100% | 0.4% | 0.4% | 0.4% |

At a two-qubit error rate of 1%, success at 4 qubits falls from 96% (noiseless) to 47%, at 6 qubits from 99.7% to 6.0% (random guessing: 1.6%), and at 8 qubits it is indistinguishable from random guessing. At a 10% two-qubit error rate it is at the random-guess level from 4 qubits; the last column, a 50% two-qubit error rate, is outside any realistic regime and shows only saturation. Real devices also suffer coherent and correlated errors that this model ignores. This demonstrates that noiseless simulator results (Section 7.2) reveal nothing about practical viability. Error correction would be

required for any non-trivial Grover instance, adding orders of magnitude in qubit overhead.

## 7.4 Classical Baseline Honesty: Goemans-Williamson vs QAOA

To illustrate why our classical baselines are insufficient for advantage claims, we set QAOA MaxCut against the Goemans-Williamson (GW) approximation guarantee [7] on three small weighted graphs:

| Graph | Nodes | Edges | Optimal Cut (brute force) | GW Guarantee (≥0.878×OPT) | Cuts Enumerated ($2^n$) |
|---|---|---|---|---|---|
| Triangle | 3 | 3 | 2.2 | ≥1.93 | 8 |
| Square+diag | 4 | 5 | 4.0 | ≥3.51 | 16 |
| Pentagon+diags | 5 | 7 | 5.4 | ≥4.74 | 32 |

GW was not run: the guarantee column is 0.878 × OPT, computed from the brute-force optimum, and bounds the expected cut of GW's randomized rounding for non-negative weights. At these sizes ($n \leq 5$) exhaustive enumeration evaluates at most 32 cuts, so it finds the optimum instantly and neither GW nor QAOA can be compared meaningfully against it. The GW guarantee, and any comparison with it, only matters at sizes where exact methods become impractical, far beyond these instances. For reference, depth-1 QAOA on 3-regular graphs is guaranteed an approximation ratio of only 0.6924 [14], below GW's 0.878. **Claiming QAOA advantage against brute-force at n=3 is meaningless** this is precisely the kind of inflated claim that our maturity gate framework is designed to prevent.

## 7.5 Lessons Learned

1. **Compiled code ≠ correct quantum code**: Analytical Q# code that runs classical math with a token qubit allocation compiles, passes CI, and produces plausible output but performs no quantum computation. Fifteen of our twenty implementations were initially such placeholders. Gate-count auditing is essential, and it is not enough: the five QPE kernels had the gates QPE needs but only diagonal ones, and the quantum-walk shift ignored its coin, and Shor's order finding had a multiplier right for two of its bases and an inverse QFT that returned a uniform register. All of them ran and passed CI for months, and all failed at once when their sampled outputs were compared with exact simulation of the circuits they claimed to be. A second audit found the same pattern in the archived kernels: a QAE reflection applied in the wrong order, two HHL kernels that were not HHL, and a swap-test encoding that dropped signs.

2. **Encoding bugs persist through validation**: A Lorentzian PDF masquerading as a log-normal distribution in the QAE implementation survived multiple development cycles because the circuit compiled and produced numbers in a plausible range. Only systematic comparison against the classical baseline revealed the error.

3. **Mock estimates previously created false confidence**: Uniform mock resource estimates across all problems suggested comparable resource requirements. Real Quantum Resource Estimator profiles now reveal a range of about 430×: from 1,746 physical qubits (QEC repetition code) to 754,715 (QPE on the materials-discovery model). Explicit T-gate counts are small (QAE: 15, HHL: 28 and 40, Shor: 9), but that does not mean the kernels avoid magic-state distillation. Every arbitrary-angle rotation must be synthesized from T states and every Toffoli consumes a CCZ state, so with QRE v3's round-based distillation factory, the only factory these estimates use ( `make_isa_query` in `tooling/estimator_config.py` composes the surface code with `RoundBasedFactory` ), the rotation-only QAOA kernels devote 71-74% of their physical qubits to T factories, the rotation-dominated QPE estimates 90-94%, and the QAE kernel's cost is set by its 3,713 rotations and 10,687 Toffolis rather than its 15 T gates. These shares belong to that factory model. Magic state cultivation, which grows a T state inside a surface-code patch and was proposed and simulated by Gidney, Shutty and Jones [25], is not modelled. It has since been run on a superconducting processor, reaching a magic-state fidelity of 0.9999(1) while keeping 8% of attempts [26], and Gidney attributes part of the qubit reduction in his RSA-2048 estimate to "allocating less space to magic state distillation by using magic state cultivation" [27]. A cultivation scenario beside the round-based estimates is tracked in #296.

4. **Process discipline has value even without quantum advantage**: The standardized structure, automated checks, and maturity gates caught real bugs and prevented inflated claims which is the framework's actual contribution.

5. **Presence checks let evidence go stale**: The policy check confirmed that a declared stage met its minimum and that two README sections existed; it never looked at the evidence Stage C requires. Five problems kept Stage C labels after their kernels were rewritten, and three held them without valid calibration statistics (Section 8, limitation 8), and nothing failed. Binding each ensemble to hashes of the sources it ran turned staleness into a failing check, and regenerating the ensembles exposed the walk defect: its zero spread had been read as a parsing problem.

6. **A test must be able to fail at its own parameters**: The QAE kernel's only sanity test prepared a uniform superposition with H, which is its own inverse, at an amplitude of 1/2, where the reversed reflection and the missing sign both leave

the output unchanged; it passed for months. The Trotter export used angles so small that a doubled coupling passed, and a fixed simulator seed makes every shot in a run identical, which produced a confident IQAE interval that excluded the true amplitude. Each new check in this revision was run against the old code, or against a deliberately broken variant, and seen to fail before it was trusted.

## 8. Limitations (Critical)

These limitations are fundamental to interpreting this work, not merely areas for improvement:

1. **No quantum advantage is claimed or demonstrated.** All problems are solved trivially by classical computers. The quantum implementations exist to validate the framework methodology, not to demonstrate quantum utility.

2. **No result comes from quantum hardware.** Results come from noiseless simulation, the H2-1E noise-model emulator, the Rigetti QVM and a syntax checker. Zero-variance results (100% QEC correction, exact QAOA optimality) are artifacts of simulation, not evidence of hardware viability. Real quantum hardware with $10^{-3}$ to $10^{-2}$ error rates would degrade results increasingly with circuit depth; the H2-1E noise model alone reduces the 4-qubit Grover kernel from an ideal 96% success to 80 of 100 shots (Section 7.2).

3. **Classical baselines are deliberately weak.** We use textbook algorithms (brute-force, naive Monte Carlo) for cross-domain standardization, and for eight problems the baseline does not compute the quantity the quantum kernel estimates (Section 4.3). Any comparative statement must acknowledge that state-of-the-art classical methods would perform far better. Quantum advantage claims have repeatedly weakened or vanished when stronger classical methods were applied [8, 9, 10].

4. **Resource estimates use the Quantum Resource Estimator v3.** All 20 problems have been profiled via `qdk.qre` against a gate-based superconducting architecture with a surface code, reporting the fewest-qubit point of the returned Pareto frontier. Physical qubit counts range from 1,746 (QEC) to 754,715 (QPE on the materials-discovery model). These are estimates under stated assumptions (a $10^{-3}$ error budget, a surface code and round-based factories), not lower or upper bounds: better compilation, codes or factories could reduce them. They describe the toy instances themselves and should not be extrapolated to production problem sizes without careful scaling analysis. Multi-model estimation across 4 hardware configurations (ns and µs gate times, each at $10^{-3}$ and $10^{-4}$ error rates) shows physical qubit requirements varying by up to 44× for the same problem

(26,092 versus 589 for Grover key search). All of that variation comes from the physical error rate: at a fixed error rate the ns and µs profiles give identical qubit counts and differ only in runtime, which is about 1,700× longer at 100 µs. Resource estimates are meaningless without the target error rate and gate time.

5. **Utility-scale screening.** Following Hoefler, Häner and Troyer [11] and Troyer's lecture series on utility-scale quantum computing [15], the project's evaluator applies six filters: a proven speedup (F1), a speedup that survives data input and output (F2), a speedup that survives error-correction overhead (F3), a naturally quantum problem (F4), a feasible crossover problem size (F5) and, for electronic structure, a preparable guiding state (F6) [17]. Eleven of the 20 problems fail at least one filter and are archived (the archival was made in April 2026 with the first five filters; F6 was added in August 2026): four with quadratic speedups (QAE ×2, Grover ×2; the trading problem was archived for the amplitude estimation it was meant to use, although its kernel samples directly), three whose speedup depends on loading classical data (HHL ×2, the swap-test classifier, for which provable kernel separations exist only for constructed datasets [24]), and four QAOA problems with no proven speedup. The nine that remain are quantum simulation problems (the five QPE problems of Table 1, a quantum walk and a Trotter simulation), Shor factoring, and the QEC repetition code, which is retained as infrastructure rather than as an application. The filters were applied to each problem class, not to its toy instance, and for two problems the difference matters. The quantum-walk kernel follows a single exciton, whose exact classical simulation already takes time polynomial in the number of sites, so its pass rests on many-body or strongly coupled open-system transport that the instance does not model. The Trotter kernel is a transverse-field Ising chain, which maps to free fermions and is classically solvable at any size, so its pass rests on interacting lattice gauge dynamics that it does not model either. Passing the filters is a screen, not evidence of advantage: even for ground-state quantum chemistry, a generic exponential quantum advantage has not been established [18], in part because preparing a state with sufficient overlap with the ground state may itself be hard, which is the condition F6 tests.

6. **Toy instances reveal nothing about asymptotic behavior.** A 2-qubit VQE or 4-qubit QAOA demonstrates circuit correctness but says nothing about how the algorithm behaves at 50, 100, or 1000 qubits. The scaling analysis in Section 7.3 illustrates how rapidly resource requirements grow.

7. **The maturity gate model is procedural, not complexity-theoretic.** It enforces process discipline but cannot compensate for unfavorable asymptotic scaling. A problem that passes all four maturity gates may still offer no practical quantum advantage if the constant factors or scaling exponents are prohibitive; the three Stage D problems, all archived by the screening above, are examples.

8. **Stage labels are declared, and only partly verified.** Each README states its own stage. Until this revision CI checked only that the declaration met a per-problem minimum and that two README sections existed (Section 5.2), and never inspected the calibration ensembles or evidence packages that Stages C and D require. Nothing therefore flagged that the calibration ensembles cited for the Stage C labels of 01, 02, 07, 14 and 17 (generated 2026-04-10 to 2026-04-12) predated the replacement of those problems' VQE hardware kernels by QPE kernels (2026-04-18), that their resource estimates still targeted the VQE routines, or that three Stage C problems lacked valid ensemble statistics: 16 had three runs, and 14 and 18 reported means, deviations and intervals of zero because their tuple- and list-valued outputs were never parsed. Following that up found worse: the QPE kernels evolved under a diagonal unitary; the quantum-walk kernel's position never depended on its coin, so once parsed its ensemble showed every run on the same site; and the Shor kernels did not find the period, while its estimate used a base with no modular arithmetic to estimate. All of these were rewritten and tested against exact simulation, every ensemble was regenerated, and CI now fails when the ensemble behind a Stage C or D label is missing, incomplete, or was computed from sources other than the current ones. The rest of what each stage requires, such as fairness reviews and scaling analyses, is still checked for presence only. That gap mattered for the QAE evidence package: its measured ensembles and fairness review came from the faulty kernel, and the review compared quantum runs at one threshold with a classical baseline at another, on a different distribution. Both are now superseded (Section 7.1). The noise study was regenerated for the current kernels on 2026-09-26; the emulator runs have not been repeated for any rewritten kernel.

## 9. Future Work

- ~~Obtain real Azure Quantum Resource Estimator profiles~~ (completed April 2026: all 20 problems profiled)
- ~~Promote Stage D candidates~~ (completed 2026-05-06, #89: QAE, QAOA MaxCut, Grover database search with scaling analysis + fairness reviews. Grover PQC was a candidate but remains at Stage B.)
- ~~Multi-model resource estimation~~ (completed April 2026: 4 qubit models × surface code = 80 estimates across all 20 problems. The Majorana profiles and floquet_code were retired because QRE v3 does not realise them for these traces.)
- ~~Evaluator agent for honest quantum/HPC/AI platform recommendation~~ (completed April 2026. Since August 2026 a deterministic router applies the six filters F1-F6 and owns the verdict, and a language model served through the Azure AI Foundry model-router writes only the explanation; model disagreement

is recorded, not applied. Since 2026-09-25 the router also owns any exponential or superpolynomial advantage class it publishes)
- ~~Code generation for recommended platform~~ (completed April 2026: Q# generator for quantum problems, Bicep workspace generator for HPC/AI Foundry/Quantum infrastructure)
- Execute small kernels on quantum hardware, not only emulators, and compare the results with the H2-1E noise model
- ~~Re-run the calibration studies for the QPE kernels and point their resource estimates at the QPE routines~~ (completed 2026-09-25, after rewriting the kernels; Section 4.2)
- Re-run the emulator studies for the rewritten QPE, quantum-walk, Shor, QAE, HHL, QAOA and swap-test kernels (Section 8, limitation 8); the noise study was regenerated on 2026-09-26
- Port the risk-analysis ensemble pipeline ( `python/analyze.py` ), which still calls the retired `dotnet` toolchain, to the `qdk` package, and regenerate its multi-instance ensembles
- Bind the other evidence artifacts to hashes of the kernels they describe, as the calibration ensembles now are, and make the CI checks required for merging, so that stale evidence blocks a merge rather than only failing a build
- Replace at least one classical baseline with a state-of-the-art competitor (Goemans-Williamson for MaxCut, importance sampling for QAE)
- Extend scaling analysis to QAOA and VQE with noise models at different error rates
- Investigate whether the maturity gate model can be extended with complexity-theoretic checks (scaling slope verification, classical hardness evidence requirements)
- Apply the cost model of Troyer's Part 6 lecture [15] (published 2026-04-28), which prices a fault-tolerant computation as its estimated runtime times an amortized cost per module-hour, alongside the evaluator's per-shot prices for today's devices

## 10. Conclusion

We have presented a software engineering framework for quantum algorithm development that prioritizes honest assessment over optimistic claims. The framework's primary value is not in the quantum implementations which are toy-scale and classically trivial but in the methodology: standardized structure, automated validation, and maturity gates designed to prevent claiming more than the evidence supports.

Of 20 implemented problems, 9 are declared at Stage C, 8 remain at Stage B, and 3 have reached Stage D (advantage evidence with filed claim contracts); for several of the Stage

C problems the supporting calibration evidence was stale or incomplete until this revision, which regenerated it and made CI fail when it goes stale again (Section 8). No problem claims demonstrated practical quantum advantage the Stage D contracts are explicitly tagged as "theoretical" or "projected" with documented residual risks. Screening the same problems against utility-scale filters archived eleven of them. Those eleven include all three Stage D problems, because maturity certifies that a claim is documented, not that it is favorable. The maturity gate model correctly identifies that emulator results on small instances, even with cross-platform validation, do not constitute evidence of quantum utility.

The practical lessons that placeholders masquerade as implementations, that encoding bugs survive validation, that mock estimates create false confidence, that presence checks let evidence go stale, that a test can be unable to fail at its own parameters are transferable to any quantum development effort. We hope the framework's insistence on honest self-assessment contributes to healthier practices in the quantum computing community.

All code, data, and tooling are available at https://github.com/WernerRall147/quantum-grand-challenges (DOI: 10.5281/zenodo.19222020).

## References


[1] T. Lubinski et al., "Application-Oriented Performance Benchmarks for Quantum Computing," IEEE Transactions on Quantum Engineering, 2023.

[2] J. Finžgar et al., "QUARK: A Framework for Quantum Computing Application Benchmarking," IEEE International Conference on Quantum Computing and Engineering, 2022.

[3] A. Javadi-Abhari et al., "Quantum Computing with Qiskit," arXiv:2405.08810, 2024.

[4] M. A. Nielsen and I. L. Chuang, "Quantum Computation and Quantum Information," Cambridge University Press, 2010.

[5] J. Preskill, "Quantum Computing in the NISQ era and beyond," Quantum, 2018.

[6] R. Babbush et al., "Focus beyond quadratic speedups for error-corrected quantum advantage," PRX Quantum, 2021.

[7] M. Goemans and D. Williamson, "Improved approximation algorithms for maximum cut and satisfiability problems using semidefinite programming," JACM, 1995.

[8] F. Pan, K. Chen, and P. Zhang, "Solving the sampling problem of the Sycamore quantum circuits," Physical Review Letters 129, 090502, 2022 (arXiv:2111.03011).

[9] J. Tindall, M. Fishman, E. M. Stoudenmire, and D. Sels, "Efficient tensor network simulation of IBM's Eagle kicked Ising experiment," PRX Quantum 5, 010308, 2024 (arXiv:2306.14887).

[10] E. Tang, "A quantum-inspired classical algorithm for recommendation systems," Proceedings of the 51st ACM Symposium on Theory of Computing (STOC), 2019 (arXiv:1807.04271).

[11] T. Hoefler, T. Häner, and M. Troyer, "Disentangling hype from practicality: on realistically achieving quantum advantage," Communications of the ACM 66(5), 82-87, 2023 (arXiv:2307.00523).

[12] D. P. DiVincenzo, "The physical implementation of quantum computation," Fortschritte der Physik 48, 771-783, 2000 (arXiv:quant-ph/0002077).

[13] M. Grassl, B. Langenberg, M. Roetteler, and R. Steinwandt, "Applying Grover's algorithm to AES: quantum resource estimates," Post-Quantum Cryptography (PQCrypto 2016), LNCS 9606, 29-43, 2016 (arXiv:1512.04965).

[14] E. Farhi, J. Goldstone, and S. Gutmann, "A quantum approximate optimization algorithm," arXiv:1411.4028, 2014.

[15] M. Troyer, "Building the Modern Quantum Architecture" lecture series, Parts 1-9, Microsoft Quantum, 2025-2026, https://quantum.microsoft.com/en-us/insights/industry-insights/quantum-architecture-series.

[16] J. A. Smolin, G. Smith, and A. Vargo, "Oversimplifying quantum factoring," Nature 499, 163-165, 2013 (arXiv:1301.7007).

[17] M. Mörchen, G. H. Low, T. Weymuth, H. Liu, M. Troyer, and M. Reiher, "Classification of electronic structures and state preparation for quantum computation of reaction chemistry," arXiv:2409.08910, 2024.

[18] S. Lee et al., "Evaluating the evidence for exponential quantum advantage in ground-state quantum chemistry," Nature Communications 14, 1952, 2023 (arXiv:2208.02199).

[19] M. E. Beverland et al., "Assessing requirements to scale to practical quantum advantage," arXiv:2211.07629, 2022.

[20] E. F. Dumitrescu et al., "Cloud quantum computing of an atomic nucleus," Physical Review Letters 120, 210501, 2018 (arXiv:1801.03897).

[21] G. Brassard, P. Høyer, M. Mosca, and A. Tapp, "Quantum amplitude amplification and estimation," Contemporary Mathematics 305, 53-74, 2002 (arXiv:quant-ph/0005055).

[22] D. Grinko, J. Gacon, C. Zoufal, and S. Woerner, "Iterative quantum amplitude estimation," npj Quantum Information 7, 52, 2021 (arXiv:1912.05559).

[23] A. W. Harrow, A. Hassidim, and S. Lloyd, "Quantum algorithm for linear systems of equations," Physical Review Letters 103, 150502, 2009 (arXiv:0811.3171).

[24] Y. Liu, S. Arunachalam, and K. Temme, "A rigorous and robust quantum speed-up in supervised machine learning," Nature Physics 17, 1013-1017, 2021 (arXiv:2010.02174).

[25] C. Gidney, N. Shutty, and C. Jones, "Magic state cultivation: growing T states as cheap as CNOT gates," arXiv:2409.17595v1, 2024.

[26] E. Rosenfeld, C. Gidney, G. Roberts, et al., "Magic state cultivation on a superconducting quantum processor," arXiv:2512.13908v1, 2025.

[27] C. Gidney, "How to factor 2048 bit RSA integers with less than a million noisy qubits," arXiv:2505.15917v1, 2025.

## Appendix A: Problem Registry

Stages are those recorded in `docs/objective-kpis.json`; status is the outcome of the utility-scale screening in Section 8. Qubit ranges are counted as in Table 1.

| # | Domain | Algorithm | Qubits | Stage | Status |
|---|---|---|---|---|---|
| 01 | Hubbard Model | VQE, then QPE | 2-14 | C | Active |
| 02 | Catalysis | VQE, then QPE | 2-12 | C | Active |
| 03 | Risk Analysis | QAE and IQAE | 5-14 | D | Archived |
| 04 | Linear Solvers | HHL | 5-7 | B | Archived |
| 05 | MaxCut | QAOA | 3 | D | Archived |
| 06 | Trading | Direct sampling (planned: QAE) | 3 | B | Archived |
| 07 | Drug Discovery | VQE, then QPE | 2-12 | C | Active |
| 08 | Protein Folding | QAOA (toy QUBO) | 4 | B | Archived |
| 09 | Factorization | Shor | 8 | C | Active |
| 10 | Cryptography | Grover | 3-5 | B | Archived |
| 11 | Machine Learning | Swap Test | 5 | B | Archived |
| 12 | Optimization | QAOA (toy QUBO) | 4 | B | Archived |

| # | Domain | Algorithm | Qubits | Stage | Status |
|---|---|---|---|---|---|
| 13 | Climate | HHL (2×2 toy) | 5-7 | B | Archived |
| 14 | Materials | VQE, then QPE | 2-12 | C | Active |
| 15 | Database Search | Grover | 4-12 | D | Archived |
| 16 | Error Correction | QEC | 5 | C | Active |
| 17 | Nuclear Physics | VQE, then QPE | 2-12 | C | Active |
| 18 | Photovoltaics | Quantum Walk | 3 | C | Active |
| 19 | QCD | Trotter | 2-4 | C | Active |
| 20 | Space Planning | QAOA (toy QUBO) | 4 | B | Archived |